Student 7 **DUDEKULA CHAND BASHA** 192424218 RD 2 / 2 submitted

Q1. Smart University Payroll Management System

Score: 25.00 **TC: 3/3 passed**

CODE

```
import java.util.*;

public class Main {

    static String nextNonEmptyLine(Scanner sc) {
        while (sc.hasNextLine()) {
            String s = sc.nextLine().trim();
            if (!s.isEmpty())
                return s;
        }
        return "";
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(nextNonEmptyLine(sc));

        Employee[] emp = new Employee[n];

        for (int i = 0; i < n; i++) {

            String type = nextNonEmptyLine(sc);

            int id = Integer.parseInt(nextNonEmptyLine(sc));
            String name = nextNonEmptyLine(sc);
            String dept = nextNonEmptyLine(sc);
            double basicSalary = Double.parseDouble(nextNonEmptyLine(sc));

            if (type.equals("TeachingFaculty")) {

                int teachingHours = Integer.parseInt(nextNonEmptyLine(sc));
                double incentivePerHour = Double.parseDouble(nextNonEmptyLine(sc));

                emp[i] = new TeachingFaculty(id, name, dept, basicSalary,
                        teachingHours, incentivePerHour);

            } else if (type.equals("TechnicalStaff"))  {

                int overtimeHours = Integer.parseInt(nextNonEmptyLine(sc));
                double overtimeRate = Double.parseDouble(nextNonEmptyLine(sc));

                emp[i] = new TechnicalStaff(id, name, dept, basicSalary,
                        overtimeHours, overtimeRate);

            } else if (type.equals("VisitingFaculty"))  {

                int lecturesHandled = Integer.parseInt(nextNonEmptyLine(sc));
                double paymentPerLecture = Double.parseDouble(nextNonEmptyLine(sc));

                emp[i] = new VisitingFaculty(id, name, dept, basicSalary,
                        lecturesHandled, paymentPerLecture);
            }
        }

        String line = nextNonEmptyLine(sc);

        if (line.equalsIgnoreCase("Search Employee ID:")) {
            line = nextNonEmptyLine(sc);
```

```
        }

        int searchId = Integer.parseInt(line);

        System.out.println("Employee Details");
System.out.println();

        double total = 0;
Employee highest = emp[0];

        for (Employee e : emp) {
e.displayEmployee();             System.out.println();

            total += e.calculateSalary();

            if (e.calculateSalary() > highest.calculateSalary()) {
highest = e;
            }
        }

        System.out.println("Highest Paid Employee");
        System.out.println("Employee ID : " + highest.employeeId);
        System.out.println("Salary : " + highest.calculateSalary());
        System.out.println();

        System.out.println("Average Salary");
System.out.printf("%.2f%n", total / n);         System.out.println();

        boolean found = false;

        for (Employee e : emp) {
if (e.employeeId == searchId) {
found = true;
                System.out.println("Employee Found");
                System.out.println("Employee ID : " + e.employeeId);
                System.out.println("Name : " + e.employeeName);
System.out.println("Salary : " + e.calculateSalary());                 break;
            }
}

        if (!found) {
            System.out.println("Employee Not Found");
        }

        sc.close();
    } }

class Employee {     protected int
employeeId;     protected String
employeeName;     protected String
department;     protected double
basicSalary;

    Employee() {
    }

    Employee(int employeeId, String employeeName, String department, double basicSalary) {
this.employeeId = employeeId;         this.employeeName = employeeName;
this.department = department;
        this.basicSalary = basicSalary;
    }
```

```
    public double calculateSalary() {
return basicSalary;
    }

    public void displayEmployee() {
        System.out.println("Employee ID : " + employeeId);
        System.out.println("Name : " + employeeName);
        System.out.println("Department : " + department);
        System.out.println("Monthly Salary : " + calculateSalary());
    } }

class TeachingFaculty extends Employee {

    int teachingHours;
double incentivePerHour;

    TeachingFaculty(int id, String name, String dept, double basicSalary,
int teachingHours, double incentivePerHour) {         super(id, name, dept,
basicSalary);         this.teachingHours = teachingHours;
this.incentivePerHour = incentivePerHour;     }

    @Override     public double calculateSalary() {         return
basicSalary + (teachingHours * incentivePerHour);
    } }

class TechnicalStaff extends Employee {

    int overtimeHours;
double overtimeRate;

    TechnicalStaff(int id, String name, String dept, double basicSalary,
int overtimeHours, double overtimeRate) {         super(id, name, dept,
basicSalary);         this.overtimeHours = overtimeHours;
this.overtimeRate = overtimeRate;     }

    @Override     public double calculateSalary() {
return basicSalary + (overtimeHours * overtimeRate);
    } }

class VisitingFaculty extends Employee {

    int lecturesHandled;
double paymentPerLecture;

    VisitingFaculty(int id, String name, String dept, double basicSalary,
int lecturesHandled, double paymentPerLecture) {         super(id, name,
dept, basicSalary);         this.lecturesHandled = lecturesHandled;
this.paymentPerLecture = paymentPerLecture;     }

    @Override
```

```
    public double calculateSalary() {
        return lecturesHandled * paymentPerLecture;
    }
}
```

INPUT

```
3

TeachingFaculty
101
Dr.Arun
CSE
50000
20
500

TechnicalStaff
102
Mr.Ravi
ECE
35000
15
300

VisitingFaculty
103
Dr.Kumar
IT
0
40
1200
102
```

OUTPUT

```
Employee Details

Employee ID : 101
Name : Dr.Arun
Department : CSE
Monthly Salary : 60000.0

Employee ID : 102
Name : Mr.Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Dr.Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr.Ravi
Salary : 39500.0
```

Q2. Smart Vehicle Insurance Claim System

Score: 25.00 **TC: 3/3 passed**

CODE

```
import java.util.*;

public class Main {

    static String nextNonEmptyLine(Scanner sc) {
        while (sc.hasNextLine()) {
            String s = sc.nextLine().trim();
            if (!s.isEmpty())
                return s;
        }
        return "";
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(nextNonEmptyLine(sc));

        Vehicle[] vehicles = new Vehicle[n];

        for (int i = 0; i < n; i++) {

            String type = nextNonEmptyLine(sc);

            String vehicleNumber = nextNonEmptyLine(sc);
            String ownerName = nextNonEmptyLine(sc);
            double insuredAmount = Double.parseDouble(nextNonEmptyLine(sc));

            if (type.equalsIgnoreCase("Car")) {
                vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
            } else if (type.equalsIgnoreCase("Bike"))  {
                vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
            } else if (type.equalsIgnoreCase("Truck"))  {
                vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
            }
        }

        String line = nextNonEmptyLine(sc);

        if (line.equalsIgnoreCase("Search Vehicle:")) {
            line = nextNonEmptyLine(sc);
        }

        String searchVehicle = line;

        Vehicle highest = vehicles[0];

        for (Vehicle v : vehicles) {
            System.out.println("Vehicle : " + v.vehicleNumber);
            System.out.println("Claim Amount : " + v.calculateClaimAmount());
            System.out.println();

            if (v.calculateClaimAmount() > highest.calculateClaimAmount()) {
                highest = v;
            }
        }

        System.out.println("Highest Claim Vehicle");
        System.out.println(highest.vehicleNumber);
```

```
        System.out.println(highest.calculateClaimAmount());
System.out.println();

        boolean found = false;

        for (Vehicle v : vehicles) {             if
(v.vehicleNumber.equals(searchVehicle)) {
found = true;
                System.out.println("Vehicle Found");
                System.out.println("Owner : " + v.ownerName);
                System.out.println("Claim Amount : " + v.calculateClaimAmount());
break;
            }
}

        if (!found) {
            System.out.println("Vehicle Not Found");
        }

        sc.close();
    } }

class Vehicle {

    protected String vehicleNumber;
protected String ownerName;     protected
double insuredAmount;

    Vehicle() {
    }

    Vehicle(String vehicleNumber, String ownerName, double insuredAmount) {
this.vehicleNumber = vehicleNumber;         this.ownerName = ownerName;
this.insuredAmount = insuredAmount;
    }

    public double calculateClaimAmount() {
return insuredAmount;
    } }

class Car extends Vehicle {

    Car(String vehicleNumber, String ownerName, double insuredAmount) {
super(vehicleNumber, ownerName, insuredAmount);     }

    @Override     public double
calculateClaimAmount() {         return
insuredAmount * 0.80;
    } }

class Bike extends Vehicle {

    Bike(String vehicleNumber, String ownerName, double insuredAmount) {
super(vehicleNumber, ownerName, insuredAmount);     }

    @Override     public double
calculateClaimAmount() {         return
insuredAmount * 0.60;
```

```
    }
}

class Truck extends Vehicle {

    Truck(String vehicleNumber, String ownerName, double insuredAmount) {
        super(vehicleNumber, ownerName, insuredAmount);
    }

    @Override
    public double calculateClaimAmount() {
        return insuredAmount * 0.90;
    }
}
```

INPUT

```
3

Car
TN10AB1234
Ramesh
800000

Bike
TN09XY5678
Suresh
100000

Truck
TN11TR9999
Logan
1500000

TN09XY5678
```

OUTPUT

```
Vehicle : TN10AB1234
Claim Amount : 640000.0

Vehicle : TN09XY5678
Claim Amount : 60000.0

Vehicle : TN11TR9999
Claim Amount : 1350000.0

Highest Claim Vehicle
TN11TR9999
1350000.0

Vehicle Found
Owner : Suresh
Claim Amount : 60000.0
```